

Laboratorio 3 - Motore di analisi CSV

Obiettivo

Evolgere il programma del Laboratorio 2 in un **motore di analisi** che, oltre a leggere e validare un file CSV, sia in grado di:

1. **Filtrare** le righe in base a condizioni specificate dall'utente
2. **Aggregare** i dati delle colonne numeriche (conteggio, somma, media, min, max)

Il laboratorio serve a consolidare:

- Definizione e implementazione di **trait** personalizzati
- **Dispatch dinamico** (`Box<dyn Trait>`) per selezionare comportamenti a runtime
- Uso di **chiusure** (closures) e funzioni di ordine superiore

Come iniziare: GitHub Classroom

Questo laboratorio utilizza **GitHub Classroom** per la distribuzione e la auto-valutazione.

Istruzioni

1. Accettate l'assignment tramite il link fornito dal docente su GitHub Classroom
2. Clonate il repository creato automaticamente sul vostro computer:
`git clone <url-del-vostro-repository>`
3. All'interno del repository troverete:
 - Un file `Cargo.toml` vuoto, a cui dovrete aggiungere le necessarie librerie con `cargo add`, per esempio: `cargo add clap --features derive`
 - Una cartella `src/` con il file `main.rs` come punto di ingresso
 - Una cartella `tests/` contenente i **test di integrazione**
 - Uno o più file CSV di esempio
4. Per compilare il progetto: `cargo build`
5. Per eseguire il programma: `cargo run -- <argomenti>`
6. Per eseguire i test forniti: `cargo test`
7. Scrivete il vostro codice all'interno della cartella `src/`, partendo dal file `main.rs`

Importante: la cartella `tests/` non va modificata in alcun modo, ma solo visionata per comprendere cosa ci si aspetta dal programma. I test forniti verificano la correttezza della vostra implementazione.

Workflow consigliato

- Fate commit frequenti e con messaggi significativi
- Eseguite `cargo test` regolarmente per verificare i progressi
- Fate push sul repository remoto per salvare il vostro lavoro:

```
git add <file-modificati>
git commit -m "descrizione della modifica"
git push
```

Specifiche funzionali

Interfaccia

Il programma deve essere eseguibile come:

```
cargo run -- <file.csv> --mode <modalità> [--column <nome_colonna>]
[--filter <espressione>]
```

Esempi validi:

```
cargo run -- data.csv --mode count
cargo run -- data.csv --mode sum --column età
cargo run -- data.csv --mode avg --column voto
cargo run -- data.csv --mode count --filter "età>25"
cargo run -- data.csv --mode avg --column voto --filter "nome=Alice"
```

Argomenti

Argomento	Obbligatorio	Descrizione
<code><file.csv></code>	Si	Percorso del file CSV da analizzare
<code>--mode</code>	Si	Modalità di aggregazione: <code>count</code> , <code>sum</code> , <code>avg</code> , <code>min</code> , <code>max</code>
<code>--column</code>	Dipende	Nome della colonna su cui operare. Obbligatorio per <code>sum</code> , <code>avg</code> , <code>min</code> , <code>max</code> . Non richiesto per <code>count</code>
<code>--filter</code>	No	Espressione di filtraggio nel formato <code>colonna>valore</code> , <code>colonna<valore</code> o <code>colonna=valore</code>

Comportamento atteso

1. Il programma legge e interpreta il CSV utilizzando la logica del Lab 2 (header + righe tipizzate)
2. Se `--filter` è specificato, filtra le righe prima dell'aggregazione
3. Applica l'aggregatore selezionato con `--mode` sulle righe (filtrate o meno)

4. Stampa il risultato nel formato specificato nella sezione [Output](#)

Modalità di aggregazione

Tutte le operazioni numeriche lavorano internamente con valori `f64`. Per le colonne intere e decimali, il valore numerico è quello del campo stesso. Per le colonne di **testo**, il valore numerico utilizzato è la **lunghezza della stringa** (numero di caratteri). Questo rende tutte le colonne compatibili con tutte le modalità di aggregazione.

- **count**: conta il numero di righe (non richiede `--column`)
- **sum**: somma i valori della colonna specificata
- **avg**: calcola la media dei valori della colonna specificata
- **min**: trova il valore minimo nella colonna specificata
- **max**: trova il valore massimo nella colonna specificata

Filtraggio

L'espressione di filtro ha il formato `colonna<operatore>valore` dove l'operatore può essere:

- `=` : uguaglianza
- `>` : maggiore di
- `<` : minore di

Esempio: `--filter "età>25"` seleziona solo le righe in cui il campo `età` è maggiore di 25.

Il filtraggio deve essere implementato tramite una **chiusura** (closure) che viene passata come parametro alla funzione di analisi.

Output

Il formato di output è **fisso** e deve essere rispettato esattamente. Ogni riga è una coppia chiave-valore separata da `:` (due punti e spazio).

Quando `--column` e `--filter` sono presenti:

```
mode: avg
column: voto
filter: età>25
result: 8.75
rows_analyzed: 2
```

Quando `--filter` non è presente:

```
mode: avg
column: voto
result: 8.75
rows_analyzed: 4
```

Quando `--column` non è presente (solo per `count`):

`mode: count`

`result: 4`

`rows_analyzed: 4`

Regole di formattazione del risultato

- Per `count`: il risultato è un intero (es. `4`)
- Per `sum`, `avg`, `min`, `max`: il risultato è sempre un valore `f64`, formattato con almeno una cifra decimale quando il numero è intero (es. `100.0`, `25.0`), altrimenti con le cifre decimali necessarie (es. `7.75`, `3.5`)
- Se non ci sono righe da analizzare (dopo il filtraggio o file con solo header), il risultato è `NaN` per `avg`, `min`, `max`; `0` per `count`; `0.0` per `sum`

Le righe di output che non sono applicabili (es. `column` quando si usa `count` senza `--column`, oppure `filter` quando `--filter` non è stato specificato) **non devono essere stampate**.

Architettura

Il trait `Aggregator`

Il cuore del laboratorio è la definizione di un **trait** che astrae il concetto di aggregazione:

```
pub trait Aggregator {  
    /// Aggiorna lo stato interno con un nuovo valore numerico (f64)  
    fn update(&mut self, value: f64);  
    /// Restituisce il risultato dell'aggregazione come stringa  
    fn result(&self) -> String;  
    /// Restituisce il nome della modalità (es. "count", "sum", ...)  
    fn mode_name(&self) -> &'static str;  
}
```

Tutti i valori vengono convertiti in `f64` **prima** di essere passati all'aggregatore: per i campi `Integer` e `Float` si usa il valore numerico diretto, per i campi `Text` si usa la lunghezza della stringa. In questo modo il trait non dipende dal tipo `Field` e lavora esclusivamente con valori numerici.

Implementazioni richieste

Dovete implementare le seguenti struct, ciascuna con `impl Aggregator`:

1) `Count`

- restituisce il conteggio come stringa (es. `"42"`)

2) Sum

- restituisce la somma formattata (es. "100.0", "31.5"). Se nessun valore è stato processato, restituisce "0.0"

3) Average

- restituisce la media come stringa. Se il contatore è 0, restituisce "NaN"

4) Min/Max

- restituisce il minimo/massimo come stringa, oppure "NaN" se nessun valore è stato processato

Dispatch dinamico e factory function

Poiché la scelta dell'aggregatore avviene **a runtime** (l'utente passa `--mode` da riga di comando), non è possibile conoscere il tipo concreto a compile-time. La soluzione è usare i **trait object**: un puntatore a un oggetto il cui tipo concreto è sconosciuto, ma che implementa il trait `Aggregator`.

Implementate una **factory function** che, dato il nome della modalità, restituisce l'aggregatore corrispondente impacchettato in un `Box<dyn Aggregator>`:

```
fn make_aggregator(mode: &str) -> Result<Box<dyn Aggregator>> {
    match mode {
        "count" => Ok(Box::new(Count::new())),
        "sum"   => Ok(Box::new(Sum::new())),
        "avg"   => Ok(Box::new(Average::new())),
        "min"   => Ok(Box::new(Min::new())),
        "max"   => Ok(Box::new(Max::new())),
        _      => Err(/* errore: modalità non valida */)
    }
}
```

In `main`, questa funzione viene usata per ottenere l'aggregatore scelto dall'utente. Da quel momento in poi, si lavora con `Box<dyn Aggregator>`, chiamando i metodi del trait senza conoscere il tipo concreto:

```
let mut aggregator = make_aggregator(&args.mode)?;
// ... filtraggio e iterazione ...
aggregator.update(field.to_f64());
// ... alla fine ...
println!("result: {}", aggregator.result());
```

Filtraggio con chiusure

Il filtraggio delle righe deve essere implementato utilizzando le **chiusure**. A partire dall'espressione testuale (es. "età>25"), costruite una chiusura che possa essere usata con, ad esempio, la funzione `.filter()` degli iteratori, come segue:

```
let filter = make_filter(&expression, &csv.headers());  
csv.rows().iter()  
    .filter(|row| filter(row))  
    .map(|row| row[column_index].to_f64())  
    .for_each(|val| aggregator.update(val));
```

Nota: ragionate su cosa viene catturato e come (per riferimento o per valore/spostamento) dalla chiusura. Poiché la chiusura deve sopravvivere alla funzione `make_filter`, i valori catturati devono essere posseduti dalla chiusura stessa (parola chiave `move`, oppure cattura di valori che implementano `Copy`).

Se `--filter` non è specificato, la pipeline deve comunque funzionare: potete usare una chiusura che restituisce sempre `true`, oppure gestire i due casi separatamente.

Struttura del progetto

Struttura minima consigliata (evoluzione del Lab 2):

`main.rs`

- Parsing degli argomenti CLI
- Lettura e parsing del CSV
- Costruzione di aggregatore e filtro e loro esecuzione
- Stampa del risultato nel formato richiesto
- Gestione degli errori

`cli.rs, csv.rs`

- Ripresi ed espansi con metodi necessari dal Lab 2

`aggregator.rs`

- Trait `Aggregator`
- Struct `Count, Sum, Average, Min, Max` con relative `impl Aggregator`

`analysis.rs`

- `make_aggregator(mode: &str) -> Result<Box<dyn Aggregator>>`
- `make_filter(...) -> ...`

Gestione errori (requisito obbligatorio)

Il programma **non deve andare in panic** in uso normale.

Devono essere gestiti:

- File inesistente o non leggibile
- File vuoto o con solo header
- `--mode` mancante o non valido (stampare le modalita disponibili)
- `--column` mancante quando richiesto dalla modalita
- `--column` che riferisce una colonna inesistente
- `--filter` con espressione malformata o colonna inesistente
- Errori di parsing del CSV (come nel Lab 2)
- Errori di I/O

In caso di errore:

- Stampare un messaggio descrittivo su `stderr`
- Terminare con exit code diverso da 0

È vietato usare `unwrap()` o `expect()` su input proveniente dall'utente o dal filesystem.

Criteri di autovalutazione

35% - Correttezza funzionale

- Aggregazioni corrette (count, sum, avg, min, max)
- Filtraggio corretto
- Formato di output rispettato
- Tutti i test forniti passano (`cargo test`)

25% - Robustezza

- Nessun panic
- Errori gestiti correttamente con messaggi utili
- Exit code appropriato
- Gestione di casi limite (nessuna riga dopo filtraggio, colonna non numerica, ecc.)

25% - Architettura e uso dei concetti

- Trait `Aggregator` definito e implementato correttamente
- Uso di trait object (`Box<dyn Aggregator>`) nella factory function
- Uso appropriato di chiusure per il filtraggio
- Uso di catene di iteratori per processare i dati

15% - Processo di sviluppo

- Repository Git con messaggi significativi, visibili con `git log`
- Deve compilare eseguendo `cargo build`
- Deve funzionare eseguendo `cargo run -- <file.csv> --mode count`

Obiettivo didattico

Alla fine di questo laboratorio dovrete essere in grado di:

- Definire un **trait** personalizzato e implementarlo su più struct diverse
- Usare **trait object** (`Box<dyn Trait>`) per selezionare implementazioni a runtime
- Comprendere il concetto di **dispatch dinamico** (vtable) e quando usarlo
- Creare e utilizzare **chiusure** che catturano variabili dall'ambiente circostante
- Restituire chiusure da funzioni usando `Box<dyn Fn(...)>`
- Comporre pipeline di elaborazione dati con **iteratori** (`map`, `filter`, `for_each`)
- Progettare un programma modulare e estensibile, con un'architettura a trait che anticipa pattern comuni nella programmazione concorrente